
Redes avanzadas 1

NetDevOps - Automatización Híbrida e IPsec

NOMBRE: Dylan Palavecino, Cristobal Figueroa, Bairon Chiuacura

CARRERA: Ingeniería en Telecomunicaciones

ASIGNATURA: Redes Avanzadas 1

PROFESOR: Felipe Henrriquez

FECHA: 30/04/2026

Introducción

El presente informe tiene como objetivo implementar una solución de automatización de red basada en principios NetDevOps, integrando tecnologías tradicionales y modernas para el despliegue de una VPN IPsec.

Se desarrolla una topología en GNS3 que simula un entorno real compuesto por una sede central, un proveedor de servicios (ISP) y una sucursal, permitiendo establecer comunicación segura entre redes privadas mediante cifrado.

Para ello, se utiliza Python como herramienta principal de automatización, empleando librerías como Netmiko para dispositivos Cisco y Requests para la interacción con API REST en MikroTik.

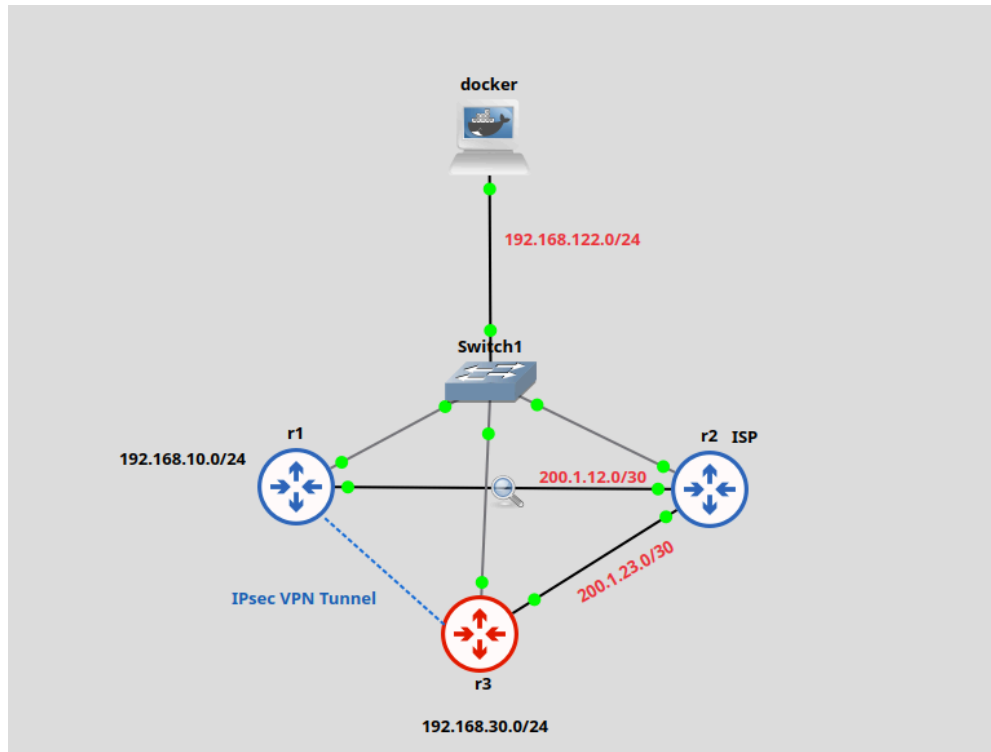
Objetivo general

Implementar y automatizar la configuración de una VPN IPsec utilizando herramientas de programación y protocolos de red.

Objetivos específicos

- Configurar una topología de red en GNS3
- Implementar conectividad entre routers
- Automatizar la configuración mediante Python
- Validar el funcionamiento del túnel VPN

Topología



La topología está compuesta por tres routers:

- R1: Sede Central
- R2: ISP
- R3: Sucursal MikroTik
- Switch: Permite conectar 3 routers al contenedor docker por medio de SSH (netmiko, API Rest)
- Servidor docker: Es donde ejecutamos nuestro script y donde se almacena la automatización

Enlaces

URL del Repositorio GitHub:

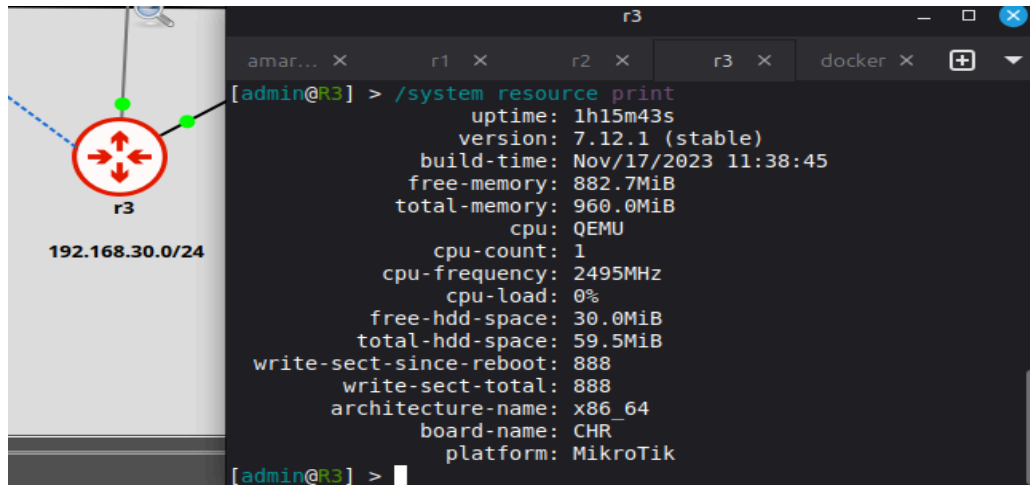
<https://github.com/Amaroo00/REDES-AVANZADAS1>

URL de Descarga del Video de Presentación:

https://drive.google.com/file/d/18_VEr919_cX5H25BqMd-oYs2XSVFhLNp/view?usp=sharing

Para este proyecto se seleccionó MikroTik RouterOS CHR como el equipo destinado a la automatización moderna vía API. Esta elección se fundamenta en los siguientes puntos clave

1. Soporte Nativo REST: RouterOS (v7.12.1) cuenta con una API REST madura y documentada que permite prescindir de librerías intermedias, enviando configuraciones directamente mediante el formato universal JSON.



2. Eficiencia de Recursos en Laboratorio: MikroTik CHR es extremadamente ligero en consumo de CPU y RAM dentro de GNS3, lo que asegura total estabilidad en la topología.

r3 configuration

General settings | HDD | CD/DVD | Network | Advanced | Usage

Name: r3

RAM: 1024 MB

vCPUs: 1

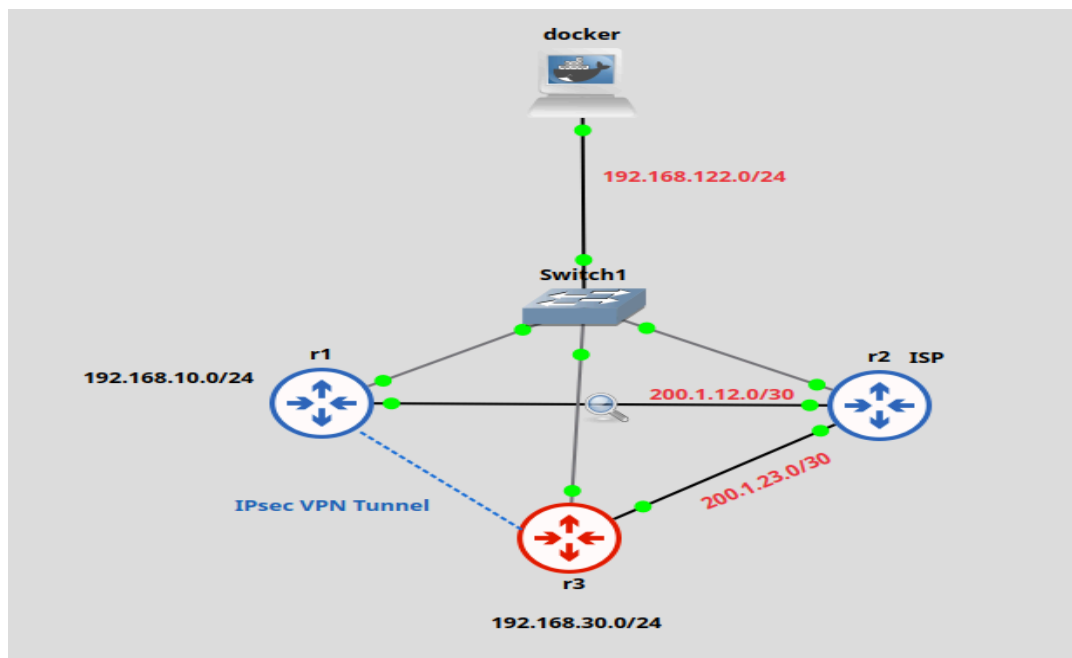
Qemu binary: /usr/bin/qemu-system-x86_64 (v8.2.2)

Boot priority: HDD

On close: Power off the VM

Console type: telnet ☐ Auto start console

3. Interoperabilidad IPsec: Posee una robusta compatibilidad técnica con los estándares abiertos de IPsec Fase 1 y Fase 2, ESP, permitiendo acoplarse de manera transparente y segura contra el nodo Cisco IOSv de la Sede Central (R1).



Desarrollo del Script `App.py` y Cumplimiento de Criterios

El script principal App.py fue desarrollado siguiendo estándares de código limpio basados en PEP8, incorporando una estructura modular mediante funciones como configurar cisco, configurar r3 api y verificar. Esta organización permite separar responsabilidades, facilitando la lectura, mantenimiento y escalabilidad del código.

El flujo del programa comienza con la definición de credenciales y dispositivos de red, seguido de la ejecución de funciones de automatización para Cisco y MikroTik. Finalmente, se incluye una etapa de verificación que permite validar el estado de la red y la correcta operación de la VPN.

Además, el script implementa múltiples bloques try/except junto con el uso de la librería logging, lo que permite capturar errores como fallos de conexión, problemas de autenticación o errores en la ejecución de comandos. Esto evita la detención abrupta del programa y proporciona información clara para diagnóstico, garantizando un Entorno de Desarrollo Eficiente

1. Netmiko en Router CISCO

```
# Configuración R1 (Sede Central)
with ConnectHandler(**R1) as net:
    net.enable()
    logging.info("Configurando interfaces y VPN IPsec en R1...")
    config_commands = [
        "interface Gi0/1", "ip address 200.1.12.1 255.255.255.252", "no shut",
        "interface Lo10", "ip address 192.168.10.1 255.255.255.0",
        "ip route 0.0.0.0 0.0.0.0 200.1.12.2",
        "crypto isakmp policy 10",
        "encryption aes 128", "hash sha", "authentication pre-share", "group 2",
        "crypto isakmp key cisco1234 address 200.1.23.2",
        "crypto ipsec transform-set TS esp-aes esp-sha-hmac",
        "access-list 100 permit ip 192.168.10.0 0.0.0.255 192.168.30.0 0.0.0.255",
        "crypto map VPN 10 ipsec-isakmp",
        "set peer 200.1.23.2",
        "set transform-set TS",
        "match address 100",
        "interface Gi0/1", "crypto map VPN"
    ]
    net.send_config_set(config_commands)
    logging.info("Cisco R1 y R2 configurados exitosamente.")
```

“Se observa el uso de Netmiko para automatizar la ejecución de comandos en dispositivos Cisco mediante SSH”

2. Bloque de configuración IPsec R1

```
# Configuración R1 (Sede Central)
with ConnectHandler(**R1) as net:
    net.enable()
    logging.info("Configurando interfaces y VPN IPsec en R1...")
    config_commands = [
        "interface Gi0/1", "ip address 200.1.12.1 255.255.255.252", "no shut",
        "interface Lo10", "ip address 192.168.10.1 255.255.255.0",
        "ip route 0.0.0.0 0.0.0.0 200.1.12.2",
        "crypto isakmp policy 10",
        "encryption aes 128", "hash sha", "authentication pre-share", "group 2",
        "crypto isakmp key cisco1234 address 200.1.23.2",
        "crypto ipsec transform-set TS esp-aes esp-sha-hmac",
        "access-list 100 permit ip 192.168.10.0 0.0.0.255 192.168.30.0 0.0.0.255",
        "crypto map VPN 10 ipsec-isakmp",
        "set peer 200.1.23.2",
        "set transform-set TS",
        "match address 100",
        "interface Gi0/1", "crypto map VPN"
    ]
```

“Se muestra el bloque automatizado de configuración de la VPN IPsec, incluyendo Fase 1, Fase 2 y asociación del tráfico interesante.”

Imagen de prueba

```
--- ISAKMP SA ---
IPv4 Crypto ISAKMP SA
dst          src          state          conn-id status
200.1.12.1   200.1.23.2   QM_IDLE        1004 ACTIVE
200.1.12.1   200.1.23.2   MM_NO_STATE    1003 ACTIVE (deleted)
200.1.12.1   200.1.23.2   QM_IDLE        1002 ACTIVE

IPv6 Crypto ISAKMP SA

--- IPSEC SA ---

interface: GigabitEthernet0/2
Crypto map tag: VPN, local addr 200.1.13.1
```

3. Configuración de R2 CISCO ISP

```
# Configuración R2 (Router ISP)
with ConnectHandler(**R2) as net:
    net.enable()
    logging.info("Configurando rutas estáticas base en R2 (ISP)...")
    net.send_config_set([
        "interface Gi0/1", "ip address 200.1.12.2 255.255.255.252", "no shut",
        "interface Gi0/2", "ip address 200.1.23.1 255.255.255.252", "no shut",
        "ip route 200.1.12.0 255.255.255.252 Gi0/1",
        "ip route 200.1.23.0 255.255.255.252 Gi0/2"
    ])
])
```

“Se estableció el ruteo base inyectando las rutas estáticas simulando el tránsito de la red pública internet”

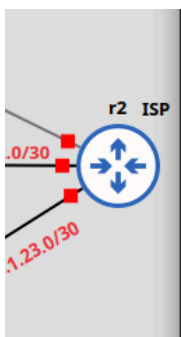
Se implementa manejo de errores mediante bloques try/except

```
try:
    # Configuración R2 (Router ISP)
    with ConnectHandler(**R2) as net:
        net.enable()
        logging.info("Configurando rutas estáticas base en R2 (ISP)...")
        net.send_config_set([
            "interface Gi0/1", "ip address 200.1.12.2 255.255.255.252", "no shut",
            "interface Gi0/2", "ip address 200.1.23.1 255.255.255.252", "no shut",
            "ip route 200.1.12.0 255.255.255.252 Gi0/1",
            "ip route 200.1.23.0 255.255.255.252 Gi0/2"
        ])

    # Configuración R1 (Sede Central)
    with ConnectHandler(**R1) as net:
        net.enable()
        logging.info("Configurando interfaces y VPN IPsec en R1...")
        config_commands = [
            "interface Gi0/1", "ip address 200.1.12.1 255.255.255.252", "no shut",
            "interface Lo10", "ip address 192.168.10.1 255.255.255.0",
            "ip route 0.0.0.0 0.0.0.0 200.1.12.2",
            "crypto isakmp policy 10",
            "encryption aes 128", "hash sha", "authentication pre-share", "group 2",
            "crypto isakmp key cisco1234 address 200.1.23.2",
            "crypto ipsec transform-set TS esp-aes esp-sha-hmac",
            "access-list 100 permit ip 192.168.10.0 0.0.0.255 192.168.30.0 0.0.0.255",
            "crypto map VPN 10 ipsec-isakmp",
            "set peer 200.1.23.2",
            "set transform-set TS",
            "match address 100",
            "interface Gi0/1", "crypto map VPN"
        ]
        net.send_config_set(config_commands)
        logging.info("Cisco R1 y R2 configurados exitosamente.")
except Exception as e:
    logging.error(f"Error en automatización Cisco: {e}")
```

Se implementa manejo de errores mediante bloques try/except, lo que permite capturar fallos de conexión o errores en la ejecución de comandos, evitando que el script se detenga abruptamente.

ejemplo:

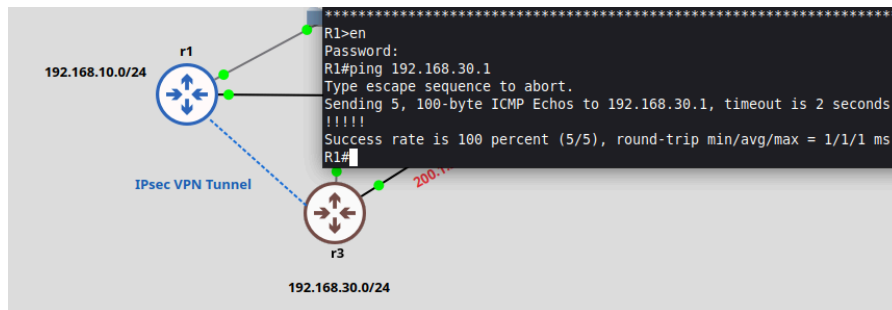


```
=====
AUTOMATIZACIÓN NETDEVOPS - DESPLIEGUE VPN
=====
2026-04-28 04:29:54,491 - INFO - >>> INICIANDO AUTOMATIZACIÓN CISCO
2026-04-28 04:30:04,496 - ERROR - Error en automatización Cisco: TCP connection to device failed.

Common causes of this problem are:
1. Incorrect hostname or IP address.
2. Wrong TCP port.
3. Intermediate firewall blocking access.

Device settings: cisco_ios 192.168.122.20:22
```

4. Ping de R1 a R3



“La conectividad exitosa entre redes demuestra el correcto funcionamiento del túnel VPN.”

5. Automatización API REST y Formato Payload JSON en MikroTik

La configuración del router MikroTik R3 Sucursal se realizó mediante un enfoque moderno basado en el uso de API REST, complementado con acceso inicial vía SSH. Este enfoque híbrido permite combinar la flexibilidad del acceso tradicional con las ventajas de automatización estructurada que ofrecen las APIs.

En una primera etapa, se establece una conexión SSH utilizando la librería `netmiko`, la cual permite preparar el dispositivo habilitando servicios necesarios y eliminando configuraciones previas que podrían generar conflictos. Entre estas acciones se incluyen la activación del servicio web, eliminación de políticas IPsec existentes y configuración de direcciones IP en las interfaces correspondientes.

Posteriormente, se implementa la automatización mediante API REST utilizando la librería `requests`, enviando solicitudes HTTP hacia el endpoint del router. Este método permite configurar de manera estructurada los distintos componentes necesarios para el establecimiento del túnel VPN.

Configuración de parámetros IPsec vía API

A través de solicitudes HTTP (PUT y PATCH), se configuran los siguientes elementos:

→ **Perfil IPsec fase 1:**

Se definen los algoritmos de cifrado (AES-128), autenticación (SHA1) y grupo Diffie-Hellman, los cuales permiten establecer un canal seguro inicial con el peer remoto.

→ **Propuesta IPsec fase 2:**

Se especifican los mecanismos de cifrado y autenticación que se utilizarán para proteger el tráfico de datos dentro del túnel.

→ **Peer IPsec:**

Se configura la dirección del dispositivo remoto (R1), estableciendo el punto de conexión para la VPN.

→ **Identidad:**

Se define el método de autenticación mediante clave pre-compartida o pre-shared key, permitiendo la validación entre ambos extremos del túnel.

→ **Política IPsec:**

Se establece el tráfico interesante que será cifrado, definiendo como origen la red 192.168.30.0/24 y como destino la red 192.168.10.0/24, además de los parámetros de encapsulación y direcciones de seguridad.

El script procesa las respuestas del router en formato JSON utilizando la librería `json`, lo que permite interpretar los resultados de cada solicitud enviada. En particular, se valida la creación de políticas IPsec mediante la verificación del código de estado HTTP y la extracción del identificador (`.id`) de la política creada. En caso de error, el sistema registra la respuesta entregada por el router, facilitando la detección de problemas en la automatización.

1. Configuración vía API

```
requests.put(f"{R3_API}/ip/ipsec/peer", json={"name": "peer1", "address": "200.1.12.1", "exchange-mode": "main", "profile": "default"}, auth=R3_AUTH)
requests.put(f"{R3_API}/ip/ipsec/identity", json={"peer": "peer1", "auth-method": "pre-shared-key", "secret": "cisco1234"}, auth=R3_AUTH)
```

“Se observa el envío de solicitudes HTTP hacia el router MikroTik para la creación de políticas IPsec mediante API REST.”

2. JSON

```
# --- PARSEO DE RESPUESTA JSON ---
if res.status_code in [200, 201]:
    data = json.loads(res.text) # Convertir string JSON a diccionario
    logging.info(f"ÉXITO: Política IPsec creada en MikroTik con ID: {data.get('.id')}")
```

“La respuesta del router es procesada en formato JSON, permitiendo validar la correcta aplicación de configuraciones.”

3. Configuración previa vía SSH

```
with ConnectHandler(**R1) as net:
    net.enable()
    print("\n--- Hostname ---")
    print(net.send_command("show running-config | include hostname"))
    print("\n--- Interfaces IP ---")
    print(net.send_command("show ip interface brief"))
    print("\n--- Rutas ---")
    print(net.send_command("show ip route"))
    print("\n--- ISAKMP SA ---")
    print(net.send_command("show crypto isakmp sa"))
    print("\n--- IPSEC SA ---")
    print(net.send_command("show crypto ipsec sa"))
    print("\n--- Crypto Map ---")
    print(net.send_command("show crypto map"))
    print("\n--- Ping a R3 ---")
    res = net.send_command("ping 192.168.30.1 source 192.168.10.1 repeat 10", read_timeout=30)
    print(res)
    if "!!!" in res:
        print("\n" + "*" * 45)
        print(" RESULTADO: VPN OPERATIVA (TRÁNSITO VIA ISP) ")
        print("*" * 45)
```

“Se realiza una preparación inicial del router mediante SSH antes de aplicar configuraciones vía API.”

4. Verificación en MikroTik

```
[admin@R3] > /ip ipsec policy print
Flags: T - TEMPLATE; X - DISABLED; A - ACTIVE; * - DEFAULT
Columns: PEER, TUNNEL, SRC-ADDRESS, DST-ADDRESS, PROTOCOL, ACTION, LEVEL, PH2-COUNT
#  PEER  TUNNEL  SRC-ADDRESS  DST-ADDRESS  PROTOCOL  ACTION  LEVEL  PH2-COUNT
0  TX*                ::/0          ::/0          all
1  A peer1  yes    192.168.30.0/24  192.168.10.0/24  all    encrypt  require    2
[admin@R3] > /ip ipsec peer print
Flags: X - disabled; D - dynamic; R - responder
0  name="peer1" address=200.1.12.1/32 profile=default exchange-mode=main
    send-initial-contact=yes
```

“Se verifica la correcta creación de los parámetros IPsec en el dispositivo MikroTik.”

5. Captura de tráfico en wireshark

esp						
No.	Time	Source	Destination	Protocol	Length	Info
29	69.748987	200.1.12.1	200.1.23.2	ESP	182	ESP (SPI=0x081041ec)
30	69.749446	200.1.23.2	200.1.12.1	ESP	182	ESP (SPI=0x62411e5c)
31	69.750147	200.1.12.1	200.1.23.2	ESP	182	ESP (SPI=0x081041ec)
32	69.750539	200.1.23.2	200.1.12.1	ESP	182	ESP (SPI=0x62411e5c)
33	69.751484	200.1.12.1	200.1.23.2	ESP	182	ESP (SPI=0x081041ec)
34	69.751947	200.1.23.2	200.1.12.1	ESP	182	ESP (SPI=0x62411e5c)
35	69.752672	200.1.12.1	200.1.23.2	ESP	182	ESP (SPI=0x081041ec)
36	69.753147	200.1.23.2	200.1.12.1	ESP	182	ESP (SPI=0x62411e5c)
37	69.753930	200.1.12.1	200.1.23.2	ESP	182	ESP (SPI=0x081041ec)
38	69.754428	200.1.23.2	200.1.12.1	ESP	182	ESP (SPI=0x62411e5c)
39	69.755098	200.1.12.1	200.1.23.2	ESP	182	ESP (SPI=0x081041ec)
40	69.755454	200.1.23.2	200.1.12.1	ESP	182	ESP (SPI=0x62411e5c)
41	69.756378	200.1.12.1	200.1.23.2	ESP	182	ESP (SPI=0x081041ec)
42	69.756697	200.1.23.2	200.1.12.1	ESP	182	ESP (SPI=0x62411e5c)
43	69.757377	200.1.12.1	200.1.23.2	ESP	182	ESP (SPI=0x081041ec)
44	69.757761	200.1.23.2	200.1.12.1	ESP	182	ESP (SPI=0x62411e5c)
45	69.758782	200.1.12.1	200.1.23.2	ESP	182	ESP (SPI=0x081041ec)
46	69.759120	200.1.23.2	200.1.12.1	ESP	182	ESP (SPI=0x62411e5c)
47	69.759805	200.1.12.1	200.1.23.2	ESP	182	ESP (SPI=0x081041ec)
48	69.760149	200.1.23.2	200.1.12.1	ESP	182	ESP (SPI=0x62411e5c)

Frame 29: 182 bytes on wire (1456 bits), 182 bytes captured on interface 0 Ethernet II, Src: 0c:a8:3c:17:00:01 (0c:a8:3c:17:00:01), Dst: 0c:a8:3c:17:00:02 (0c:a8:3c:17:00:02) Internet Protocol Version 4, Src: 200.1.12.1, Dst: 200.1.23.2 Encapsulating Security Payload	<pre> 0000 0c 04 45 d5 00 01 0c a8 3c 17 00 01 08 00 45 00 0010 00 a8 03 b0 00 00 ff 32 04 0e c8 01 0c 01 c8 01 0020 17 02 08 19 41 ec 00 00 00 01 20 b8 89 ab 35 ae 0030 e8 d0 6d 4b bb 1e da 14 dd 0f e3 1c 9b cb 57 d0 0040 c9 68 d6 13 7b b6 b8 5b be cf e4 de 78 cd d9 1c 0050 bc 1b f5 ac d1 9c 8b eb 98 d4 6b ef 42 72 98 0f 0060 5e 2b 9d 7f 3a 43 08 e2 6d 49 93 04 ac b4 db e0 0070 e1 de 5e 16 01 54 d4 77 93 44 0d 30 e6 69 df 78 0080 31 39 83 6f 76 78 cc 6d 91 5d c1 5c 3a 4e dd b3 0090 f2 be 03 0c e8 fd 0a 5c 98 62 72 66 a1 de f5 2e 00a0 a4 1d a0 b2 0d ff 7c 0e f9 7d e0 15 cc 4f 0e 52 00b0 f1 e7 0c 97 64 b2 </pre>
---	--

isakmp						
No.	Time	Source	Destination	Protocol	Length	Info
7	26.638074	200.1.23.2	200.1.12.1	ISAKMP	118	Informational
8	26.719316	200.1.23.2	200.1.12.1	ISAKMP	134	Informational
9	26.720094	200.1.12.1	200.1.23.2	ISAKMP	134	Informational
12	29.001930	200.1.23.2	200.1.12.1	ISAKMP	390	Identity Protection (Main Mode)
13	29.002829	200.1.12.1	200.1.23.2	ISAKMP	150	Identity Protection (Main Mode)
14	29.017699	200.1.23.2	200.1.12.1	ISAKMP	278	Identity Protection (Main Mode)
15	29.019492	200.1.12.1	200.1.23.2	ISAKMP	346	Identity Protection (Main Mode)
16	29.034626	200.1.23.2	200.1.12.1	ISAKMP	118	Identity Protection (Main Mode)
17	29.035314	200.1.12.1	200.1.23.2	ISAKMP	150	Identity Protection (Main Mode)
18	29.036235	200.1.23.2	200.1.12.1	ISAKMP	214	Quick Mode
19	29.037770	200.1.12.1	200.1.23.2	ISAKMP	246	Quick Mode
20	29.038431	200.1.23.2	200.1.12.1	ISAKMP	102	Quick Mode

“Primero se observa tráfico ISAKMP donde se realiza la negociación de la VPN mediante Main Mode y Quick Mode. Posteriormente, una vez establecidas las asociaciones de seguridad, se inicia el tráfico ESP cifrado entre los extremos, lo que confirma que el túnel IPsec está operativo.”

Evidencias de Funcionamiento

1. ejecución de python App.py dentro de la consola del contenedor GNS3

```
root@docker:/app# python App.py

=====
AUTOMATIZACIÓN NETDEVOPS - DESPLIEGUE VPN
=====

2026-04-28 05:25:41,630 - INFO - >>> INICIANDO AUTOMATIZACIÓN CISCO
2026-04-28 05:25:41,633 - INFO - Connected (version 1.99, client Cisco-1.25)
2026-04-28 05:25:41,901 - INFO - Auth banner: b"\r\n*****\r\n* IOSv is strictly limited to use for evaluation, demonst
n and IOS *\\r\\n* education. IOSv is provided as-is and is not supported by Cisco's *\\
Technical Advisory Center. Any use or disclosure, in whole or in part, *\\r\\n* of the IOSv S
re or Documentation to any third party for any *\\r\\n* purposes is expressly prohibite
ept as otherwise authorized by *\\r\\n* Cisco in writing.
*\\r\\n*****"
2026-04-28 05:25:41,903 - INFO - Authentication (password) successful!
2026-04-28 05:25:42,141 - INFO - Configurando rutas estáticas base en R2 (ISP)...
2026-04-28 05:25:42,391 - INFO - Connected (version 1.99, client Cisco-1.25)
2026-04-28 05:25:42,658 - INFO - Auth banner: b"\r\n*****\r\n* IOSv is strictly limited to use for evaluation, demonst
n and IOS *\\r\\n* education. IOSv is provided as-is and is not supported by Cisco's *\\
Technical Advisory Center. Any use or disclosure, in whole or in part, *\\r\\n* of the IOSv S
re or Documentation to any third party for any *\\r\\n* purposes is expressly prohibite
ept as otherwise authorized by *\\r\\n* Cisco in writing.
*\\r\\n*****"
2026-04-28 05:25:42,659 - INFO - Authentication (password) successful!
2026-04-28 05:25:42,844 - INFO - Configurando interfaces y VPN IPsec en R1...
2026-04-28 05:25:43,306 - INFO - Cisco R1 y R2 configurados exitosamente.
2026-04-28 05:25:43,317 - INFO - >>> INICIANDO AUTOMATIZACIÓN MIKROTIK (API REST)
2026-04-28 05:25:43,322 - INFO - Connected (version 2.0, client ROSSSH)
2026-04-28 05:25:43,504 - INFO - Authentication (password) successful!
2026-04-28 05:25:44,280 - INFO - ÉXITO: Política IPsec creada en MikroTik con ID: *1000003

Automatización finalizada. Use --verify para validar.
root@docker:/app#
```

Usamos el comando (python [App.py](#) --verify) para verificar todo

```
root@docker:/app# python App.py --verify

=====
AUTOMATIZACIÓN NETDEVOPS - DESPLIEGUE VPN
=====

=====
VERIFICACIÓN COMPLETA DE LA RED VPN
=====

=====
VERIFICACIÓN R1 (Cisco)
=====
2026-04-28 05:29:42,670 - INFO - Connected (version 1.99, client Cisco-1.25)
2026-04-28 05:29:42,937 - INFO - Auth banner: b"\r\n*****
as-is and is not supported by Cisco's *\r\n* Technical Advisory Center. Any use or disc
except as otherwise authorized by *\r\n* Cisco in writing.
2026-04-28 05:29:42,938 - INFO - Authentication (password) successful!

--- Hostname ---
hostname R1

--- Interfaces IP ---
Interface IP-Address OK? Method Status Protocol
GigabitEthernet0/0 192.168.122.10 YES NVRAM up up
GigabitEthernet0/1 200.1.12.1 YES NVRAM up up
GigabitEthernet0/2 200.1.13.1 YES NVRAM down down
GigabitEthernet0/3 unassigned YES NVRAM administratively down down
Loopback10 192.168.10.1 YES NVRAM up up
NVI0 192.168.122.10 YES unset up up

--- Rutas ---
Codes: L - local, C - connected, S - static, R - RIP, M - mobile, B - BGP
D - EIGRP, EX - EIGRP external, O - OSPF, IA - OSPF inter area
N1 - OSPF NSSA external type 1, N2 - OSPF NSSA external type 2
E1 - OSPF external type 1, E2 - OSPF external type 2
i - IS-IS, su - IS-IS summary, L1 - IS-IS level-1, L2 - IS-IS level-2
ia - IS-IS inter area, * - candidate default, U - per-user static route
o - ODR, P - periodic downloaded static route, H - NHRP, l - LISP
a - application route
+ - replicated route, % - next hop override, p - overrides from PfR

Gateway of last resort is 200.1.12.2 to network 0.0.0.0

S* 0.0.0.0/0 [1/0] via 200.1.12.2
192.168.10.0/24 is variably subnetted, 2 subnets, 2 masks
C 192.168.10.0/24 is directly connected, Loopback10
L 192.168.10.1/32 is directly connected, Loopback10
192.168.122.0/24 is variably subnetted, 2 subnets, 2 masks
C 192.168.122.0/24 is directly connected, GigabitEthernet0/0
L 192.168.122.10/32 is directly connected, GigabitEthernet0/0
200.1.12.0/24 is variably subnetted, 2 subnets, 2 masks
C 200.1.12.0/30 is directly connected, GigabitEthernet0/1
L 200.1.12.1/32 is directly connected, GigabitEthernet0/1
200.1.23.0/30 is subnetted, 1 subnets
S 200.1.23.0 [1/0] via 200.1.12.2
```



```
--- ISAKMP SA ---
IPv4 Crypto ISAKMP SA
dst          src          state          conn-id status
200.1.12.1   200.1.23.2   QM_IDLE       1005 ACTIVE
200.1.12.1   200.1.23.2   QM_IDLE       1002 ACTIVE

IPv6 Crypto ISAKMP SA

--- IPSEC SA ---

interface: GigabitEthernet0/2
  Crypto map tag: VPN, local addr 200.1.13.1

  protected vrf: (none)
  local ident (addr/mask/prot/port): (192.168.10.0/255.255.255.0/0/0)
  remote ident (addr/mask/prot/port): (192.168.30.0/255.255.255.0/0/0)
  current peer 200.1.13.2 port 500
    PERMIT, flags={origin_is_acl,}
    #pkts encaps: 0, #pkts encrypt: 0, #pkts digest: 0
    #pkts decaps: 0, #pkts decrypt: 0, #pkts verify: 0
    #pkts compressed: 0, #pkts decompressed: 0
    #pkts not compressed: 0, #pkts compr. failed: 0
    #pkts not decompressed: 0, #pkts decompress failed: 0
    #send errors 0, #recv errors 0

    local crypto endpt.: 200.1.13.1, remote crypto endpt.: 200.1.13.2
    plaintext mtu 1500, path mtu 1500, ip mtu 1500, ip mtu idb GigabitEthernet0/2
    current outbound spi: 0x0(0)
    PFS (Y/N): N, DH group: none

  inbound esp sas:

  inbound ah sas:

  inbound pcp sas:

  outbound esp sas:

  outbound ah sas:

  outbound pcp sas:

  local crypto endpt.: 200.1.13.1, remote crypto endpt.: 200.1.23.2
  plaintext mtu 1500, path mtu 1500, ip mtu 1500, ip mtu idb GigabitEthernet0/2
  current outbound spi: 0x0(0)
  PFS (Y/N): N, DH group: none
```

```
inbound ah sas:

inbound pcp sas:

outbound esp sas:

outbound ah sas:

outbound pcp sas:

interface: GigabitEthernet0/1
  Crypto map tag: VPN, local addr 200.1.12.1

protected vrf: (none)
local ident (addr/mask/prot/port): (192.168.10.0/255.255.255.0/0/0)
remote ident (addr/mask/prot/port): (192.168.30.0/255.255.255.0/0/0)
current peer 200.1.23.2 port 500
  PERMIT, flags={origin is acl,}
  #pkts encaps: 20, #pkts encrypt: 20, #pkts digest: 20
  #pkts decaps: 20, #pkts decrypt: 20, #pkts verify: 20
  #pkts compressed: 0, #pkts decompressed: 0
  #pkts not compressed: 0, #pkts compr. failed: 0
  #pkts not decompressed: 0, #pkts decompress failed: 0
  #send errors 0, #recv errors 0

local crypto endpt.: 200.1.12.1, remote crypto endpt.: 200.1.13.2
plaintext mtu 1500, path mtu 1500, ip mtu 1500, ip mtu idb GigabitEthernet0/1
current outbound spi: 0x0(0)
PFS (Y/N): N, DH group: none

inbound esp sas:

inbound ah sas:

inbound pcp sas:

outbound esp sas:

outbound ah sas:

outbound pcp sas:

local crypto endpt.: 200.1.12.1, remote crypto endpt.: 200.1.23.2
plaintext mtu 1438, path mtu 1500, ip mtu 1500, ip mtu idb GigabitEthernet0/1
current outbound spi: 0x75ED475(123655285)
PFS (Y/N): N, DH group: none

inbound esp sas:
  spi: 0xB8C605D1(3099985361)
  transform: esp-aes esp-sha-hmac ,
  in use settings =(Tunnel, )
  conn id: 17, flow id: SW:17, sibling flags 80000040, crypto map: VPN
  sa timing: remaining key lifetime (k/sec): (4220203/1562)
  IV size: 16 bytes
  replay detection support: Y
  Status: ACTIVE(ACTIVE)
```

```
outbound ah sas:

outbound pcp sas:

--- Crypto Map ---
Crypto Map IPv4 "MAPA_VPN" 10 ipsec-isakmp
  Peer = 200.1.13.2
  Extended IP access list 100
    access-list 100 permit ip 192.168.10.0 0.0.0.255 192.168.30.0 0.0.0.255
  Security association lifetime: 4608000 kilobytes/3600 seconds
  Responder-Only (Y/N): N
  PFS (Y/N): N
  Mixed-mode : Disabled
  Transform sets={
    TS_VPN: { esp-256-aes esp-sha-hmac } ,
  }
  Interfaces using crypto map MAPA_VPN:
  Interfaces using crypto map NiStTeSt1:

Crypto Map IPv4 "VPN" 10 ipsec-isakmp
  Peer = 200.1.13.2
  Peer = 200.1.23.2
  Extended IP access list 100
    access-list 100 permit ip 192.168.10.0 0.0.0.255 192.168.30.0 0.0.0.255
  Current peer: 200.1.13.2
  Security association lifetime: 4608000 kilobytes/3600 seconds
  Responder-Only (Y/N): N
  PFS (Y/N): N
  Mixed-mode : Disabled
  Transform sets={
    TS: { esp-aes esp-sha-hmac } ,
  }
  Interfaces using crypto map VPN:
    GigabitEthernet0/2

    GigabitEthernet0/1

--- Ping a R3 ---
Type escape sequence to abort.
Sending 10, 100-byte ICMP Echos to 192.168.30.1, timeout is 2 seconds:
Packet sent with a source address of 192.168.10.1
!!!!!!
Success rate is 100 percent (10/10), round-trip min/avg/max = 1/1/1 ms

*****
RESULTADO: VPN OPERATIVA (TRÁNSITO VIA ISP)
*****
```

```
=====
VERIFICACIÓN R2 (Cisco)
=====
2026-04-28 05:29:43,904 - INFO - Connected (version 1.99, client Cisco-1.25)
2026-04-28 05:29:44,171 - INFO - Auth banner: b"\r\n*****
as-is and is not supported by Cisco's      *\r\n* Technical Advisory Center. Any use
except as otherwise authorized by      *\r\n* Cisco in writing.
2026-04-28 05:29:44,172 - INFO - Authentication (password) successful!

--- Interfaces IP ---
Interface                IP-Address      OK? Method Status          Protocol
GigabitEthernet0/0       192.168.122.20  YES NVRAM   up              up
GigabitEthernet0/1       200.1.12.2     YES NVRAM   up              up
GigabitEthernet0/2       200.1.23.1     YES NVRAM   up              up
GigabitEthernet0/3       unassigned      YES NVRAM   administratively down down

--- Rutas ---
Codes: L - local, C - connected, S - static, R - RIP, M - mobile, B - BGP
       D - EIGRP, EX - EIGRP external, O - OSPF, IA - OSPF inter area
       N1 - OSPF NSSA external type 1, N2 - OSPF NSSA external type 2
       E1 - OSPF external type 1, E2 - OSPF external type 2
       i - IS-IS, su - IS-IS summary, L1 - IS-IS level-1, L2 - IS-IS level-2
       ia - IS-IS inter area, * - candidate default, U - per-user static route
       o - ODR, P - periodic downloaded static route, H - NHRP, l - LISP
       a - application route
       + - replicated route, % - next hop override, p - overrides from PfR

Gateway of last resort is not set

S    192.168.10.0/24 [1/0] via 200.1.12.1
S    192.168.30.0/24 [1/0] via 200.1.23.2
     192.168.122.0/24 is variably subnetted, 2 subnets, 2 masks
C     192.168.122.0/24 is directly connected, GigabitEthernet0/0
L     192.168.122.20/32 is directly connected, GigabitEthernet0/0
     200.1.12.0/24 is variably subnetted, 2 subnets, 2 masks
C     200.1.12.0/30 is directly connected, GigabitEthernet0/1
L     200.1.12.2/32 is directly connected, GigabitEthernet0/1
     200.1.13.0/30 is subnetted, 1 subnets
S     200.1.13.0 [1/0] via 200.1.12.1
     200.1.23.0/24 is variably subnetted, 2 subnets, 2 masks
C     200.1.23.0/30 is directly connected, GigabitEthernet0/2
L     200.1.23.1/32 is directly connected, GigabitEthernet0/2
```

```
VERIFICACIÓN R3 (MikroTik)
=====
2026-04-28 05:29:44,605 - INFO - Connected (version 2.0, client R0SSH)
2026-04-28 05:29:44,780 - INFO - Authentication (password) successful!

--- Sistema ---
Hostname: name: R3

--- Interfaces IP ---
Columns: ADDRESS, NETWORK, INTERFACE
# ADDRESS NETWORK INTERFACE
0 192.168.122.30/24 192.168.122.0 ether1
1 192.168.30.1/24 192.168.30.0 bridge-vpn
2 200.1.23.2/30 200.1.23.0 ether2

--- Rutas ---
Flags: D - DYNAMIC; I - INACTIVE; A - ACTIVE; c - CONNECT, s - STATIC; H - HW-OFFLOADED; + - ECMP
Columns: DST-ADDRESS, GATEWAY, DISTANCE
# DST-ADDRESS GATEWAY DISTANCE
0 As + 0.0.0.0/0 200.1.23.1 1
1 As + 0.0.0.0/0 200.1.23.1 1
2 As + 0.0.0.0/0 200.1.23.1 1
3 As + 0.0.0.0/0 200.1.23.1 1
4 As + 0.0.0.0/0 200.1.23.1 1
5 As + 0.0.0.0/0 200.1.23.1 1
6 As + 0.0.0.0/0 200.1.23.1 1
7 As + 0.0.0.0/0 200.1.23.1 1
8 As + 0.0.0.0/0 200.1.23.1 1
9 As + 0.0.0.0/0 200.1.23.1 1
10 As + 0.0.0.0/0 200.1.23.1 1
11 As + 0.0.0.0/0 200.1.23.1 1
12 As + 0.0.0.0/0 200.1.23.1 1
13 As + 0.0.0.0/0 200.1.23.1 1
14 As + 0.0.0.0/0 200.1.23.1 1
15 As + 0.0.0.0/0 200.1.23.1 1
16 As + 0.0.0.0/0 200.1.23.1 1
17 As + 0.0.0.0/0 200.1.23.1 1
18 As + 0.0.0.0/0 192.168.122.1 1
19 IsH 192.168.10.0/24 200.1.13.1 1
20 IsH 192.168.10.0/24 200.1.13.1 1
21 IsH 192.168.10.0/24 200.1.13.1 1
22 IsH 192.168.10.0/24 200.1.13.1 1
23 IsH 192.168.10.0/24 200.1.13.1 1
24 IsH 192.168.10.0/24 200.1.13.1 1
25 IsH 192.168.10.0/24 200.1.13.1 1
26 IsH 192.168.10.0/24 200.1.13.1 1
27 IsH 192.168.10.0/24 200.1.13.1 1
28 IsH 192.168.10.0/24 200.1.13.1 1
29 IsH 192.168.10.0/24 200.1.13.1 1
30 IsH 192.168.10.0/24 200.1.13.1 1
31 IsH 192.168.10.0/24 200.1.13.1 1
32 IsH 192.168.10.0/24 200.1.13.1 1
33 IsH 192.168.10.0/24 200.1.13.1 1
34 IsH 192.168.10.0/24 200.1.13.1 1
35 IsH 192.168.10.0/24 200.1.13.1 1
36 IsH 192.168.10.0/24 200.1.13.1 1
DAs 192.168.30.0/24 bridge-vpn 0
```

```
--- IPSec Peers ---
Flags: X - disabled; D - dynamic; R - responder
0 name="peer1" address=200.1.12.1/32 profile=default exchange-mode=main send-initial-contact=yes

--- IPSec Policies ---
Flags: T - TEMPLATE; X - DISABLED; A - ACTIVE; * - DEFAULT
Columns: PEER, TUNNEL, SRC-ADDRESS, DST-ADDRESS, PROTOCOL, ACTION, LEVEL, PH2-COUNT
# PEER TUNNEL SRC-ADDRESS DST-ADDRESS PROTOCOL ACTION LEVEL PH2-COUNT
0 TX*
1 A peer1 yes 192.168.30.0/24 192.168.10.0/24 all encrypt require 1

=====
VERIFICACIÓN FINALIZADA
=====
root@docker:/app#
```

Con esto concluimos la verificación del script ejecutado en el contenedor

Conclusión

El presente proyecto permitió implementar de manera exitosa una solución de conectividad segura mediante una VPN IPsec entre distintas sedes, utilizando una topología simulada en GNS3 que representa un entorno de red real.

A través de la automatización con Python, se logró configurar dispositivos Cisco y MikroTik de forma eficiente, reduciendo la intervención manual y optimizando los tiempos de despliegue. La integración de tecnologías tradicionales, como la configuración por CLI mediante Netmiko en equipos Cisco, junto con enfoques modernos basados en API REST en MikroTik, evidenció la importancia de adoptar soluciones híbridas en la administración de redes actuales. Este enfoque permite mayor flexibilidad, escalabilidad y adaptación a distintos escenarios operacionales.

Asimismo, la aplicación de buenas prácticas de programación, como el diseño modular, el uso de manejo de excepciones (try/except) y la implementación de registros (logging), contribuyó a desarrollar un sistema robusto, capaz de enfrentar errores y facilitar su diagnóstico. La fase de verificación confirmó el correcto funcionamiento del túnel VPN, validando tanto el establecimiento de asociaciones de seguridad como la conectividad entre redes privadas. En conclusión, este trabajo no solo permitió comprender en profundidad el funcionamiento de las VPN IPsec, sino también destacar el rol fundamental de la automatización en la gestión de infraestructuras de red modernas, siendo una herramienta clave para mejorar la eficiencia, seguridad y confiabilidad de los sistemas.